DATA MAROC BOOTCAMPS

Module 6 · Airflow & GCP

Exercice — Airflow & GCP

construire un pipeline d'ingestion **cloud**

CONTEXTE

Tu passes un entretien technique chez Data Maroc. Pour évaluer ta capacité à construire un pipeline de données de bout en bout sur le Cloud, le recruteur te propose un cas pratique : mettre en place, sur GCP, un DAG Airflow qui va chercher des indicateurs économiques et sociaux sur l'API publique de la Banque Mondiale (World Bank API), déposer les données brutes sur Cloud Storage, puis les charger dans BigQuery pour analyse.

AIRFLOW · PARTIE A

Environnement Airflow sur GCP

1. Active les APIs nécessaires sur ton projet GCP : `composer.googleapis.com`, `storage.googleapis.com` et `bigquery.googleapis.com`.
2. Crée un environnement `Cloud Composer 3` en mode `serverless` dans la région `europe-west9` (Paris), et attends qu'il passe à l'état `Healthy`.
3. Crée un bucket Cloud Storage dédié à la zone brute, par exemple `<project>-worldbank-raw`.
4. Crée un dataset BigQuery `raw_worldbank` qui recevra les données chargées.
5. Identifie et attribue au compte de service utilisé par Composer les rôles IAM minimaux dont il a besoin pour écrire sur ton bucket et charger des données dans BigQuery — cherche du côté de Cloud Storage et de BigQuery, et n'attribue rien de plus large que nécessaire.

```
$ gcloud services enable composer.googleapis.com \
      storage.googleapis.com bigquery.googleapis.com
$ gcloud composer environments create wb-bootcamp-env \
      --location europe-west9 \
      --image-version composer-3-airflow-2.11.1-build.5
$ gsutil mb -l europe-west9 gs://<project>-worldbank-raw
$ bq mk --dataset --location europe-west9 <project>:raw_worldbank
```

AIRFLOW · PARTIE B

Le DAG d'ingestion

6. Crée un DAG `worldbank_gdp_ingestion` planifié en `@weekly`, avec une tâche `PythonOperator` nommée `extract_and_upload` qui interroge l'API World Bank pour une liste de pays et d'indicateurs.

7. Attention : l'API World Bank répond parfois en `HTTP 200` avec un message d'erreur JSON plutôt qu'une vraie erreur HTTP (indicateur invalide ou archivé). Ta fonction d'extraction doit détecter explicitement ce cas — sinon la tâche se termine « avec succès » sans avoir chargé aucune ligne.
8. La tâche `extract_and_upload` doit écrire un CSV local temporaire, puis l'uploader sur Cloud Storage avec le chemin `gs://<bucket>/worldbank/{{ ds }}/indicators.csv` (utilise la date d'exécution Airflow dans le chemin).
9. Ajoute une deuxième tâche `load_to_bigquery` (`GCSToBigQueryOperator`) qui charge ce fichier dans la table `raw_worldbank.indicators`, en mode `WRITE_TRUNCATE`.
10. Enchaîne les tâches dans le bon ordre : `extract_and_upload >> load_to_bigquery`.
11. Bonus — ajoute une tâche de contrôle qualité (ex : `BigQueryCheckOperator`) qui vérifie qu'au moins une ligne a été chargée pour chaque pays de ta liste. Que se passe-t-il pour le reste du DAG si cette tâche échoue ?

```
from airflow import DAG
from airflow.operators.python import PythonOperator
from airflow.providers.google.cloud.transfers.gcs_to_bigquery import (
    GCSToBigQueryOperator,
)

with DAG("worldbank_gdp_ingestion", schedule_interval="@weekly", ...) as dag:

    extract_and_upload = PythonOperator(
        task_id="extract_and_upload",
        python_callable=extract_and_upload_to_gcs,
    )

    load_to_bigquery = GCSToBigQueryOperator(
        task_id="load_to_bigquery",
        bucket="<project>-worldbank-raw",
        source_objects=["worldbank/{{ ds }}/indicators.csv"],
        destination_project_dataset_table="raw_worldbank.indicators",
        write_disposition="WRITE_TRUNCATE",
        source_format="CSV",
        skip_leading_rows=1,
    )

    extract_and_upload >> load_to_bigquery
```

AIRFLOW · PARTIE C

Colonnes à collecter

12. Ta table `raw_worldbank.indicators` doit respecter le schéma suivant — une ligne par (pays, indicateur, année) :

> `country_code` — `STRING` · code ISO3 du pays (ex : `MAR`).
> `country_name` — `STRING` · nom du pays.
> `indicator_code` — `STRING` · code World Bank de l'indicateur (ex : `NY.GDP.MKTP.CD`).
> `indicator_label` — `STRING` · libellé lisible de l'indicateur.
> `year` — `INT64` · année de la donnée.
> `value` — `FLOAT64` · valeur de l'indicateur.
> `load_date` — `DATE` · date d'exécution du DAG (utilisée comme partition BigQuery).

13. Utilise ces 5 indicateurs pour l'exercice : GDP (`NY.GDP.MKTP.CD`), inflation (`FP.CPI.TOTL.ZG`), population (`SP.POP.TOTL`), chômage (`SL.UEM.TOTL.ZS`) et espérance de vie (`SP.DYN.LE00.IN`) — pour ces 5 pays : `MAR, TUN, DZA, EGY, FRA`.

14. Bonus — partitionne la table BigQuery par `load_date` et clusterise-la par `country_code` : quel avantage cela apporte-t-il pour des requêtes d'analyse filtrées par pays et par période ?

RENDU

Ce que tu dois rendre

> Une capture d'écran de ton environnement Composer à l'état `Healthy`.
> Le lien vers le code source de ton DAG (repo Git).
> Une capture du résultat `SELECT * FROM raw_worldbank.indicators LIMIT 20` dans BigQuery.
> Une courte réponse écrite (2-3 phrases) aux deux questions bonus.

CONSEIL

Teste d'abord ta fonction d'extraction en local, avec un simple script Python, avant de l'intégrer dans le DAG — déboguer un DAG complet à chaque itération est beaucoup plus lent que d'itérer sur un script.